Neovim IDE マニュアル

自分用の備忘録。IntelliJ の操作感を Neovim で再現した設定の使い方。

Leader キー = `Space`

目次

1. 基本操作
2. 検索 (Telescope)
3. LSP (コード支援)
4. 補完 (nvim-cmp)
5. ファイルツリー (Neo-tree)
6. Problems / 診断 (Trouble)
7. Git
8. フォーマット (Conform)
9. デバッグ (DAP)
10. テスト (Neotest)
11. ターミナル (ToggleTerm)
12. タスク実行 (Overseer)
13. ジャンプ / ナビゲーション
14. 編集支援
15. データベース (Dadbod)
16. 言語別ガイド
17. 全キーバインド一覧
18. 管理コマンド
19. トラブルシューティング

---

1. 基本操作

ファイル操作

キー	動作
`ew`	保存 (:w)
`eq`	保存して閉じる (:wq)
`Q`	強制終了 (:q!)
`Space q`	バッファを閉じる
`Space Q`	最後に閉じたファイルを再オープン

移動

キー	動作

キー	動作
`j` / `k`	表示行単位の移動（カウント付きは論理行）
`O`	下に空行追加（ノーマルモードのまま）
`[m` / `]m`	前/次のメソッドへ（treesitter）
`[e` / `]e`	前/次のエラーへ
`[d` / `]d`	前/次の診断へ
`[c` / `]c`	前/次のGit hunkへ

編集

キー	動作
`p`	インデント調整付きペースト
`Ctrl+s`	置換テンプレート（`:%s///gc`）
`Space Enter`	検索ハイライト解除

設定

キー	動作
`Space Space R`	init.lua をリロード
`Space H`	全サイドパネルを閉じる（IntelliJ の Hide All Windows）

---

2. 検索 (Telescope)

IntelliJ の Search Everywhere に相当。

キー	IntelliJ相当	動作
`Space fa`	Search Everywhere	Telescope の全ピッカー一覧
`Space fg`	Find in Path	プロジェクト全体のテキスト検索 (live_grep)
`Space ff`	Ctrl+F	現在のバッファ内をファジー検索
`Space za`	Ctrl+Shift+N	ファイル名で検索 (find_files)
`Space o`	Ctrl+F12	ドキュメントシンボル一覧
`Space zs`	Ctrl+Alt+Shift+N	ワークスペースシンボル検索
`Space zS`	〃（動的）	ワークスペースシンボル（動的）
`Space a`	Ctrl+Shift+A	コマンドパレット
`Space zL`	Recent Files	最近開いたファイル

キー	IntelliJ相当	動作
`Space zl`	Recent Locations	ジャンプリスト
`Space zp`	Recent Projects	プロジェクト一覧

Telescope 内の操作

キー	動作
`Ctrl+j` / `Ctrl+k`	結果を上下移動
`Enter`	選択して開く
`Ctrl+x`	水平分割で開く
`Ctrl+v`	垂直分割で開く
`Ctrl+t`	新タブで開く
`Esc`	閉じる

---

3. LSP (コード支援)

ファイルを開くと自動でLSPサーバーがアタッチされる。

ナビゲーション

キー	IntelliJ相当	動作
`gd`	Ctrl+Click	定義へジャンプ
`F12`	F12	定義へジャンプ
`Shift+Click`	Shift+Click	定義へジャンプ
`gD`	—	宣言へジャンプ
`gi`	Ctrl+Alt+B	実装へジャンプ
`gr`	Alt+F7	参照一覧 (Telescope)
`K`	Ctrl+Q	ホバードキュメント
`Space k`	〃	〃（leader版）

リファクタリング

キー	IntelliJ相当	動作
`Space rn`	Shift+F6	リネーム
`Space ca`	Alt+Enter	コードアクション
`Space y`	〃	〃（Intention actions）

キー	IntelliJ相当	動作
`Space u`	Ctrl+Alt+O	import 整理
`Space p`	Ctrl+Alt+L	コード整形

確認コマンド

```
:checkhealth lspconfig  " 現在のバッファにアタッチ中のLSP確認
:LspLog          " LSPログ
:Mason           " LSPサーバーの管理画面
```

---

4. 補完 (nvim-cmp)

Insert モードで自動表示。

キー	動作
`Tab`	次の候補 / スニペット展開・ジャンプ
`Shift+Tab`	前の候補 / スニペット逆ジャンプ
`Enter`	確定
`Ctrl+Space`	手動で補完を起動
`Ctrl+e`	補完を閉じる
`Ctrl+d` / `Ctrl+u`	ドキュメントをスクロール

補完ソース（優先順）

1. `[LSP]` — LSPサーバーからの補完
2. `[Snip]` — スニペット (friendly-snippets)
3. `[Buf]` — 現在のバッファ内の単語
4. `[Path]` — ファイルパス

---

5. ファイルツリー (Neo-tree)

キー	動作
`Space .`	ツリーの表示/非表示

ツリー内の操作

キー	動作
`Enter`	ファイルを開く / ディレクトリを展開

キー	動作
`a`	新規ファイル/ディレクトリ作成
`d`	削除
`r`	リネーム
`c`	コピー
`m`	移動
`yy`	絶対パスをコピー
`Y`	相対パスをコピー
`/`	フィルター
`H`	隠しファイルの表示切替
`R`	リフレッシュ

---

6. Problems / 診断 (Trouble)

IntelliJ の Problems パネルに相当。

キー	動作
`Space xx`	ワークスペース全体の診断を表示/非表示
`Space xX`	現在のバッファの診断のみ
`Space xs`	ドキュメントシンボル一覧

Trouble パネル内

キー	動作
`Enter`	該当箇所にジャンプ
`q`	閉じる
`j` / `k`	上下移動

---

7. Git

概要操作（leader v*）

キー	IntelliJ相当	動作
`Space vc`	—	lazygit を起動
`Space lg`	—	lazygit を起動
`Space vp`	Ctrl+Shift+K	git push

キー	IntelliJ相当	動作
`Space vl`	—	git pull
`Space vd`	—	全変更の差分ビュー (DiffviewOpen)
`Space vs`	—	現在ファイルのgit履歴
`Space vb`	Annotate	コミット単位のblame表示（ポップアップ）
`Space vr`	—	`git reset --soft HEAD~1`（注意!）

Hunk 操作（Gitsigns）

キー	動作
`]c`	次のhunkへ移動
`[c`	前のhunkへ移動
`Space hs`	hunkをステージ
`Space hr`	hunkをリセット
`Space hp`	hunkをプレビュー
`Space hb`	インラインblame表示のトグル

lazygit の使い方

`Space vc` または `Space lg` で起動。ターミナルUIのGitクライアント。

キー	動作
`1`〜`5`	パネル切替 (Status/Files/Branches/Commits/Stash)
`Space`	ステージ/アンステージ
`a`	全ファイルをステージ/アンステージ
`c`	コミット
`P`	Push
`p`	Pull
`r`	リベース
`?`	ヘルプ（全キー一覧）
`q`	閉じる

---

8. フォーマット (Conform)

保存時に自動フォーマット。手動も可。

キー	動作
`Space p`	手動フォーマット（ノーマル/ビジュアル）
`:ConformInfo`	現在のフォーマッター確認

言語ごとのフォーマッター

言語	フォーマッター
Lua	stylua
Python	black
TypeScript / JavaScript	prettier
JSON / HTML / CSS / SCSS / YAML	prettier
Java	google-java-format
C#	csharpier
SQL	sqlfluff

---

9. デバッグ (DAP)

基本操作

キー	IntelliJ相当	動作
`Space dc`	Shift+F9	デバッグ開始/続行
`Space ;`	Ctrl+F8	ブレークポイント切替
`Space dx`	Ctrl+F2	デバッグ停止
`Space du`	—	DAP UIの表示/非表示
`F5`	—	続行
`F10`	F8	ステップオーバー
`F11`	F7	ステップイン
`Shift+F11`	Shift+F8	ステップアウト

デバッグの流れ

1. ブレークポイントを置く: `Space ;`
2. デバッグ開始: `Space dc` （言語のDAP設定が必要）
3. DAP UIが自動で開く（変数、コールスタック、ブレークポイント一覧）
4. `F10` / `F11` でステップ実行
5. 変数の値がコード横にインライン表示される（virtual text）
6. 停止: `Space dx`

対応言語と設定

言語	DAP アダプター	Mason パッケージ
Python	debugpy	`debugpy`
TypeScript/JS	js-debug-adapter	`js-debug-adapter`
C#	netcoredbg	`netcoredbg`
Java	java-debug-adapter	`java-debug-adapter` + `java-test`（jdtls経由）

---

10. テスト (Neotest)

キー	動作
`Space tr`	カーソル位置のテストを実行
`Space td`	カーソル位置のテストをデバッグ実行
`Space ts`	テストサマリーパネルの表示/非表示
`Space to`	テスト出力パネルの表示/非表示

対応アダプター

言語	アダプター	テストフレームワーク
Python	neotest-python	pytest
TypeScript/JS	neotest-jest	Jest
C#	neotest-dotnet	dotnet test
Java	jdtls 内蔵	JUnit（`Space gt` / `Space gT`）

---

11. ターミナル (ToggleTerm)

キー	動作
`Space tt`	フローティングターミナルの表示/非表示
`Space lg`	lazygit を起動（要インストール）

ターミナル内の操作

キー	動作
`i`	ターミナルに入力
`Esc`	ノーマルモードに戻る
`Ctrl+v`	クリップボードからペースト

---

12. タスク実行 (Overseer)

IntelliJ の Run Configurations に相当。

キー	動作
`Space r`	タスク選択して実行
`:OverseerToggle`	タスク一覧パネル
`:OverseerInfo`	Overseer情報

---

13. ジャンプ / ナビゲーション

IntelliJ の Back/Forward + EasyMotion に相当。z プレフィックス。

キー	IntelliJ相当	動作
`Space zj`	Ctrl+Alt+←	戻る (jumplist)
`Space zk`	Ctrl+Alt+→	進む (jumplist)
`Space zb`	—	最後の編集位置へ
`Space za`	Ctrl+Shift+N	ファイル検索
`Space zp`	—	プロジェクト一覧
`Space zl`	—	ジャンプリスト
`Space zL`	—	最近開いたファイル
`Space zs`	—	ワークスペースシンボル
`Space zw`	EasyMotion word	Flash ジャンプ
`Space z/`	EasyMotion search	Flash 検索ジャンプ
`Space zf`	EasyMotion find	Flash 文字ジャンプ

Flash.nvim（EasyMotion代替）

キー	動作
`s`	Flash ジャンプ（ノーマルモード）
`S`	Flash treesitter 選択

`s` を押すと文字を入力 → 候補にラベルが表示 → ラベルを押して一発ジャンプ。

---

14. 編集支援

Surround（囲み操作）

キー	動作	例
`ys{motion}"`	囲みを追加	`ysiw"` → word を `"word"` に
`cs"'`	囲みを変更	`"hello"` → `'hello'`
`ds"`	囲みを削除	`"hello"` → `hello`

Comment（コメント）

キー	動作
`gcc`	行コメント切替
`gc{motion}`	範囲コメント切替（例: `gcap` で段落）
`gc` (ビジュアル)	選択範囲をコメント切替

Autopairs

括弧・クォートが自動で閉じる。`(` → `()` にカーソルが中に。

Treesitter テキストオブジェクト

キー	動作
`af` / `if`	関数の外側/内側を選択
`ac` / `ic`	クラスの外側/内側を選択
`aa` / `ia`	引数の外側/内側を選択

---

15. データベース (Dadbod)

```
:DBUI                        " DB UI を起動
:DBUIAddConnection           " 接続を追加
```

接続文字列の例

```
" Oracle
oracle://user:pass@host:1521/service_name

" PostgreSQL
postgresql://user:pass@localhost:5432/dbname

" MySQL
mysql://user:pass@localhost:3306/dbname
```

DBUI 内で `.sql` ファイルを編集すると、接続先に対する補完が効く（vim-dadbod-completion）。

16. 言語別ガイド

Java (jdtls)

専用モジュール。Java ファイルを開くと自動で jdtls が起動。

プロジェクトルート検出: `.git`, `pom.xml`, `build.gradle`, `settings.gradle`, `.project`

キー	動作
`Space go`	import 整理
`Space gc`	コンストラクタ生成
`Space gt`	最寄りのテストメソッドを実行
`Space gT`	テストクラス全体を実行
`Space ge`	変数の抽出（ビジュアル）
`Space gm`	メソッドの抽出（ビジュアル）

確認:

```
:checkhealth lspconfig  " jdtls がアタッチされているか
:Mason          " jdtls, java-debug-adapter, java-test が ✓ か
```

TypeScript (typescript-tools.nvim)

TS/JS ファイルを開くと自動起動。

- **整形**: prettier（保存時自動）
- **リント**: eslint（保存時 EslintFixAll 自動実行）
- **責務分離**: prettier = 見た目、eslint = 規約

Python (pyright + black)

- **LSP**: pyright（型チェック = basic モード）
- **整形**: black（保存時自動）
- **デバッグ**: debugpy（`Space d` で起動）
- **テスト**: pytest（`Space tr` で実行）

C# (omnisharp + netcoredbg)

- **LSP**: omnisharp（Roslyn アナライザー有効）
- **整形**: csharpier（保存時自動）
- **デバッグ**: netcoredbg
- **テスト**: neotest-dotnet

SQL (sqls + sqlfluff)

- **LSP**: sqls（補完・定義ジャンプ）
- **整形**: sqlfluff（保存時自動）
- **DB接続**: `:DBUI` で dadbod-ui

SCSS (cssls + prettier)

- **LSP**: cssls（補完・バリデーション）
- **整形**: prettier（保存時自動）
- **Emmet**: `Ctrl+y ,` で Emmet 展開（emmet-vim）

---

17. 全キーバインド一覧

Space なし（グローバル）

キー	動作
`ew`	保存
`eq`	保存+閉じる
`Q`	強制終了
`j` / `k`	表示行移動
`O`	下に空行追加
`p` / `P`	インデント調整ペースト
`Ctrl+s`	置換テンプレート
`s`	Flash ジャンプ
`S`	Flash treesitter
`gcc`	行コメント切替
`gd`	定義ジャンプ
`F12`	定義ジャンプ
`Shift+Click`	定義ジャンプ
`gD`	宣言ジャンプ
`gi`	実装ジャンプ
`gr`	参照一覧
`K`	ホバードキュメント
`[e` / `]e`	前/次のエラー
`[d` / `]d`	前/次の診断
`[m` / `]m`	前/次のメソッド
`[c` / `]c`	前/次のgit hunk

キー	動作
`F5`	デバッグ続行
`F10`	ステップオーバー
`F11`	ステップイン
`Shift+F11`	ステップアウト

Space + 単キー

キー	動作
`Space .`	ファイルツリー
`Space a`	コマンドパレット
`Space H`	全パネルを閉じる
`Space k`	ホバードキュメント
`Space o`	ドキュメントシンボル
`Space p`	コード整形
`Space q`	バッファを閉じる
`Space Q`	最後のファイルを再オープン
`Space r`	タスク実行 (Overseer)
`Space tt`	ターミナル
`Space u`	import 整理
`Space dc`	デバッグ開始/続行
`Space dx`	デバッグ停止
`Space y`	コードアクション
`Space ;`	ブレークポイント切替
`Space Enter`	検索ハイライト解除

Space + f* (検索)

キー	動作
`Space fa`	Telescope 全ピッカー
`Space ff`	バッファ内検索
`Space fg`	プロジェクト全文検索
`Space fr`	プロジェクト置換 (Spectre)

Space + g* (生成 / Java)

キー	動作
`Space go`	import 整理 (Java)
`Space gc`	コンストラクタ生成 (Java)
`Space ge`	変数抽出 (Java, visual)
`Space gm`	メソッド抽出 (Java, visual)
`Space gt`	テストメソッド実行 (Java)
`Space gT`	テストクラス実行 (Java)

Space + v* (Git)

キー	動作
`Space vb`	blame（コミット詳細ポップアップ）
`Space vc`	lazygit
`Space vd`	Diffview
`Space vl`	pull
`Space vp`	push
`Space vr`	soft reset
`Space vs`	ファイル履歴

Space + h* (Gitsigns hunk)

キー	動作
`Space hs`	hunk ステージ
`Space hr`	hunk リセット
`Space hp`	hunk プレビュー
`Space hb`	インラインblameトグル

Space + t* (テスト)

キー	動作
`Space tr`	テスト実行
`Space td`	テストデバッグ
`Space ts`	テストサマリー
`Space to`	テスト出力

Space + x* (Trouble)

キー	動作
`Space xx`	ワークスペース診断
`Space xX`	バッファ診断
`Space xs`	シンボル

Space + z* (ジャンプ)

キー	動作
`Space za`	ファイル検索
`Space zb`	最後の編集位置
`Space zf`	Flash 文字ジャンプ
`Space zj`	戻る
`Space zk`	進む
`Space zl`	ジャンプリスト
`Space zL`	最近のファイル
`Space zp`	プロジェクト一覧
`Space zs`	ワークスペースシンボル
`Space zS`	ワークスペースシンボル (動的)
`Space zw`	Flash ジャンプ
`Space z/`	Flash 検索

Space + その他

キー	動作
`Space rn`	リネーム
`Space ca`	コードアクション
`Space du`	DAP UI 切替
`Space lg`	lazygit
`Space Space R`	設定リロード

---

18. 管理コマンド

```
:Lazy                  " プラグインマネージャー
:Lazy sync             " プラグイン更新・同期
:Lazy profile          " 起動時間プロファイル

:Mason                 " LSP/DAP/フォーマッター管理
:MasonUpdate           " Mason レジストリ更新
:MasonInstall <pkg>    " パッケージインストール

:checkhealth lspconfig  " LSP接続状態
:LspLog                " LSPログ

:ConformInfo           " フォーマッター状態

:checkhealth           " 全体の健全性チェック

:TSUpdate              " Treesitter パーサー更新
:TSInstall <lang>      " パーサー追加
```

---

19. トラブルシューティング

LSP がアタッチされない

```
:checkhealth lspconfig  " サーバーの状態確認
:Mason                  " サーバーがインストールされているか ✓ 確認
```

→ サーバーが ✗ なら `:MasonInstall <server名>`

補完が出ない

1. LSPがアタッチされているか確認 (`:checkhealth lspconfig`)
2. Insert モードで `Ctrl+Space` を押してみる
3. `:checkhealth nvim-cmp` で状態確認

フォーマットされない

```
:ConformInfo            " フォーマッターが見つかっているか確認
```

→ フォーマッターが `not found` なら `:MasonInstall <formatter名>`

jdtls が起動しない

1. `java -version` → JDK 17+ か確認
2. `:Mason` → jdtls が ✓ か
3. プロジェクトに `pom.xml` or `build.gradle` or `.git` があるか
4. `:LspLog` でエラー確認

クリップボードが動かない (WSL2)

設定は `clip.exe` + `powershell.exe` を使用。

```
:checkhealth provider   " clipboard の状態確認
```

Mason のインストールが失敗する

DNS の問題の可能性あり。`/etc/resolv.conf` が Google DNS (8.8.8.8) になっているか確認:

```
cat /etc/resolv.conf
```

プラグインがロードされない

```
:Lazy                   " プラグイン一覧で状態確認
:Lazy sync              " 再同期
```

キャッシュの問題なら:

```
rm -rf ~/.local/state/nvim/lazy/
nvim                    " 再起動で再構築
```